Introduction to Computer Architecture and Systems

Chapter 3

Introduction to Little Computer - 3: Components and ISA

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- The von Neumann Architecture
  - Structure of a von Neumann (Computation Unit + Control Unit)
  - Advantage: Universal Machine (Flexible ISA against Physical Wiring)
  - Disadvantage: The von Neumann Bottleneck (Bus with no Parallelism)
  - Workaround: Harvard Architecture (Control Unit Centered)

Cover-ups…

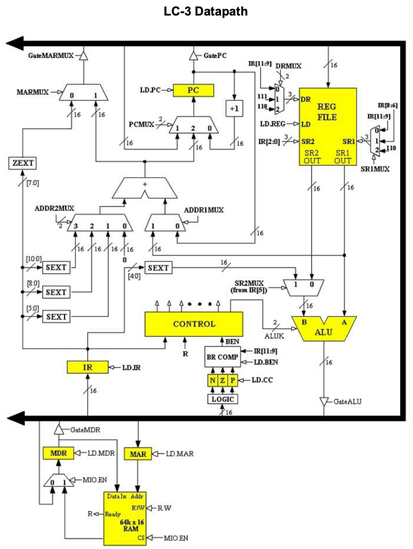
Computer Architecture and Computer Organization

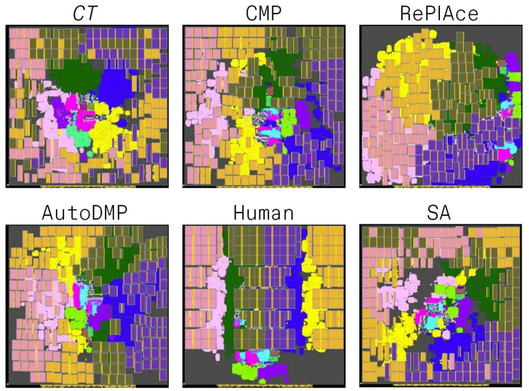
Computer Organization and Computer Architecture

- Computer Architecture: Programmer's View

Opcode	DR	SR1	[5:3]	SR2
0001	010	011	000	100
0001 0100 1100 0100				
x14C4				

- Computer Organization: Builder's View





From Logic Gates to Computer

It will be a long story…

- Logical Gates and Boolean Algebra
- Sequential Logic and Combinatory Logic
- Arithmetic Unit: Build ‘customized logic’, from Adder.
- Logic/Control Unit
- Memory/Cache: Flip Flops
- …
- We can cover them if you are interested at mathematic and hardware side!

Introducing Your New Computer

Turing Complete Guaranteed!

LITTLE COMPUTER-3



Free computer with complete customer support at CSCI260!

- **16-bit Architecture**
  Word size: 16 bits
  Memory address space
  64 K locations
- **8 General-Purpose Registers**
  R0 to R7 eusy-to-use
- **Simple Instruction Set**
  Ideal for learning assembly
- **Memory-Mapped I/O**
  Keyboard and display
  Interactive

Why LC-3?

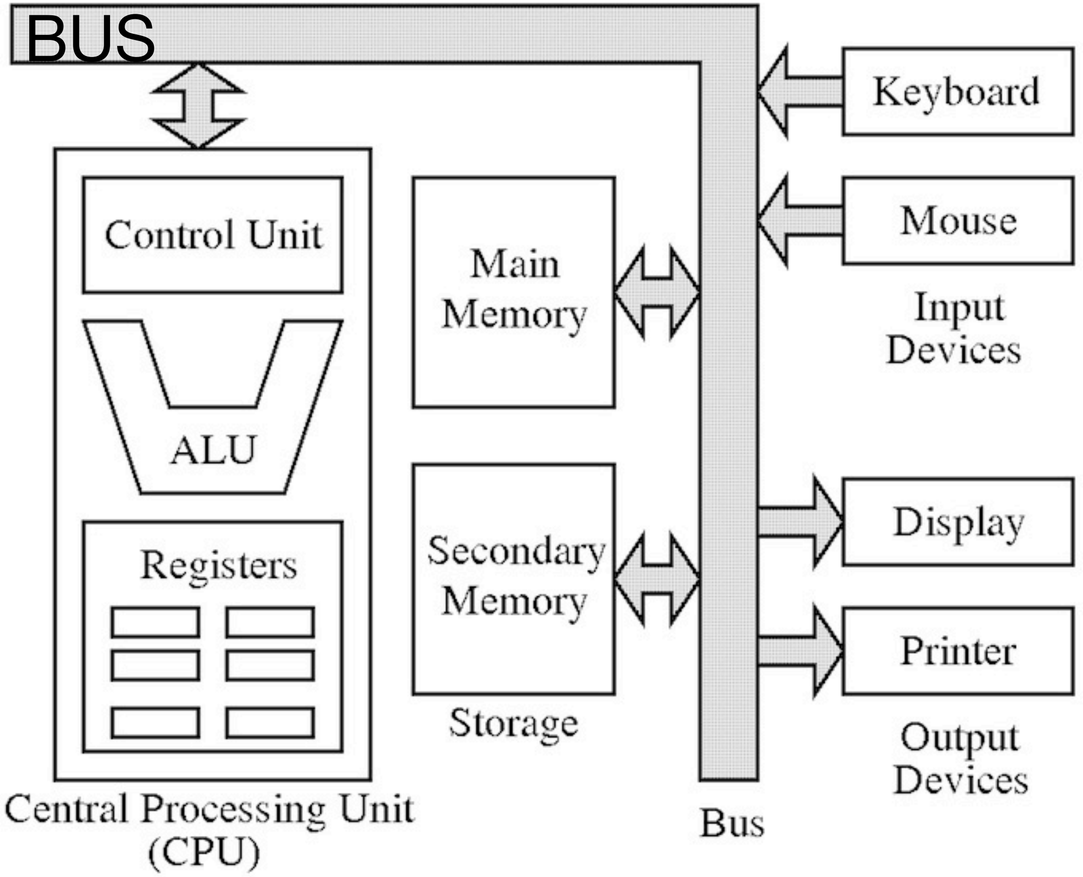
What completes a computer?

- “Turing Complete”
- “Simplest Fashion”
- “Related to all computers we are using.”

The only computer we can know EVERYTHING about,
And the only computer we need to know everything about.

Ramp it up…

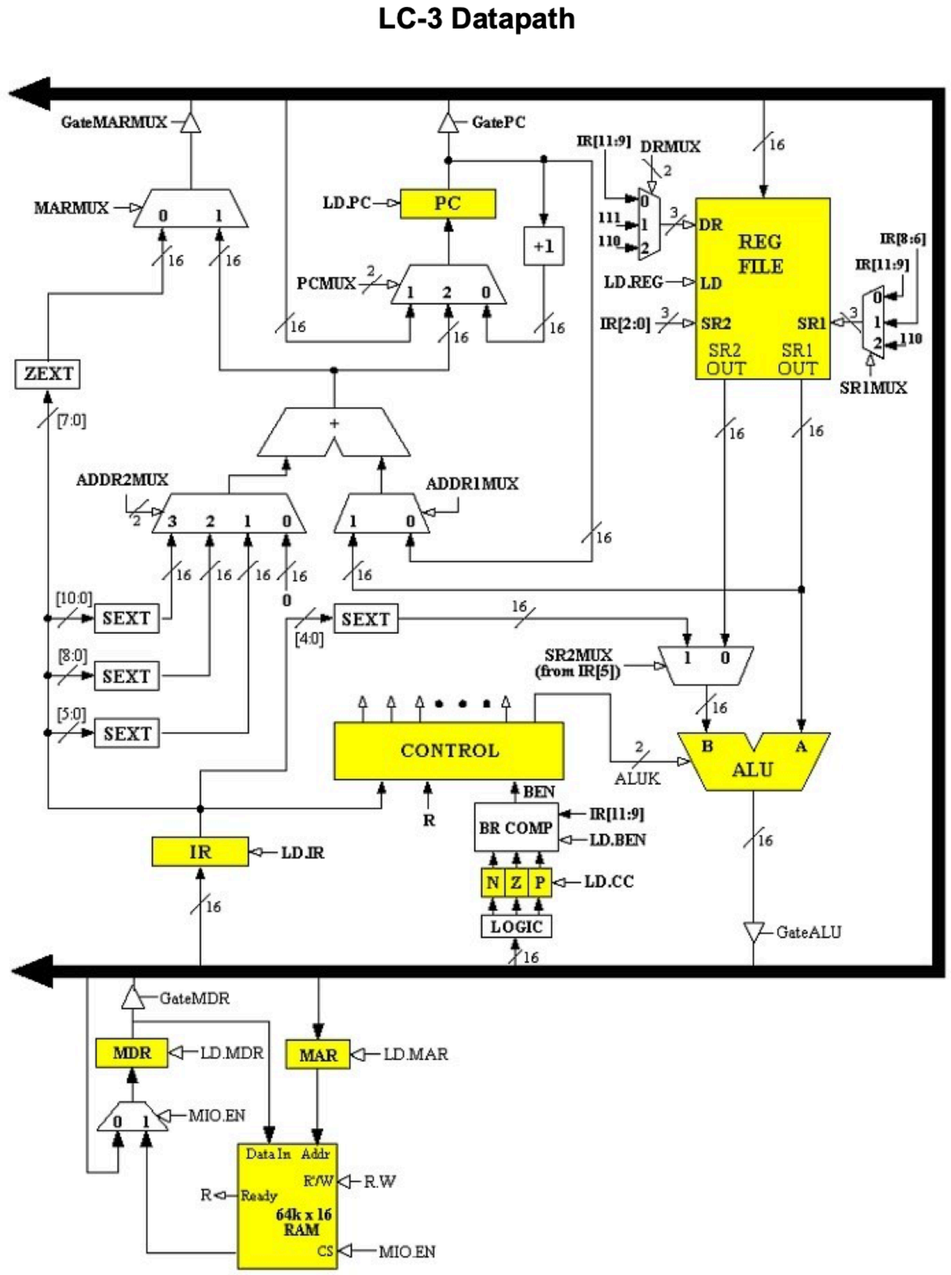
- CPU:
  + Registers: ‘Sticky Notes’
  + Control Unit: ‘Data Arrangers’
- Memory:
  + Hierarchy: ‘Fast and Slow’



Little Computer 3

First glance on ‘Data Path’

- 8 16-bit registers
- 2-byte addressable memory
- Can modify memory/registers with hex sequences



Data Types

Recall binary conversions learnt.

- 16-bit
- 2’s complement integer- immediate numbers
- Unsigned integer - registers and memory addresses

Registers

Example: R0

Registers

R0: x0000

x0000

heximal = 0000

Binary: 0000 0000 0000 0000

R0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

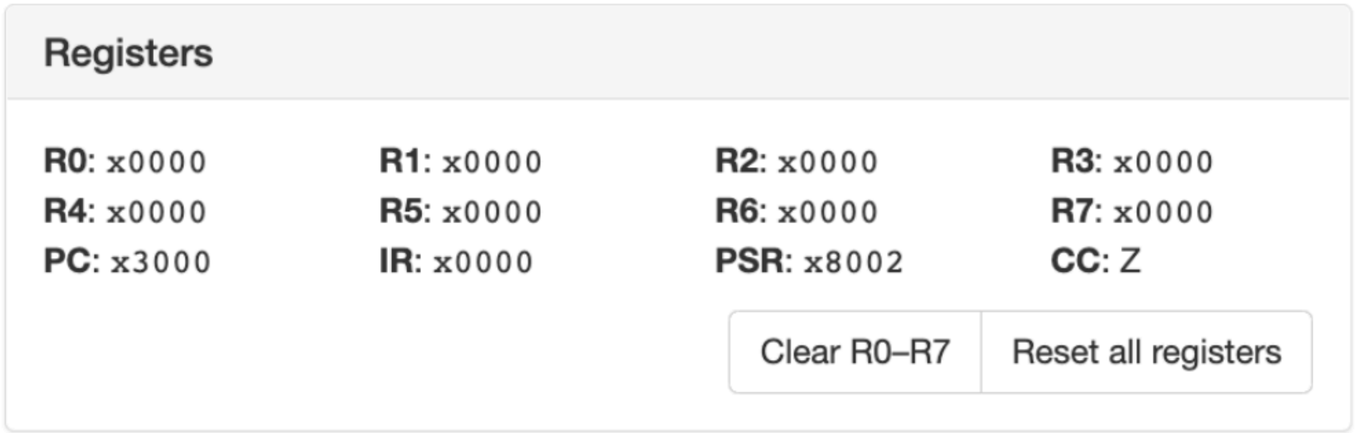
- how many bits to uniquely identify a register?

log_2states(8selections) = 3 bits

Registers

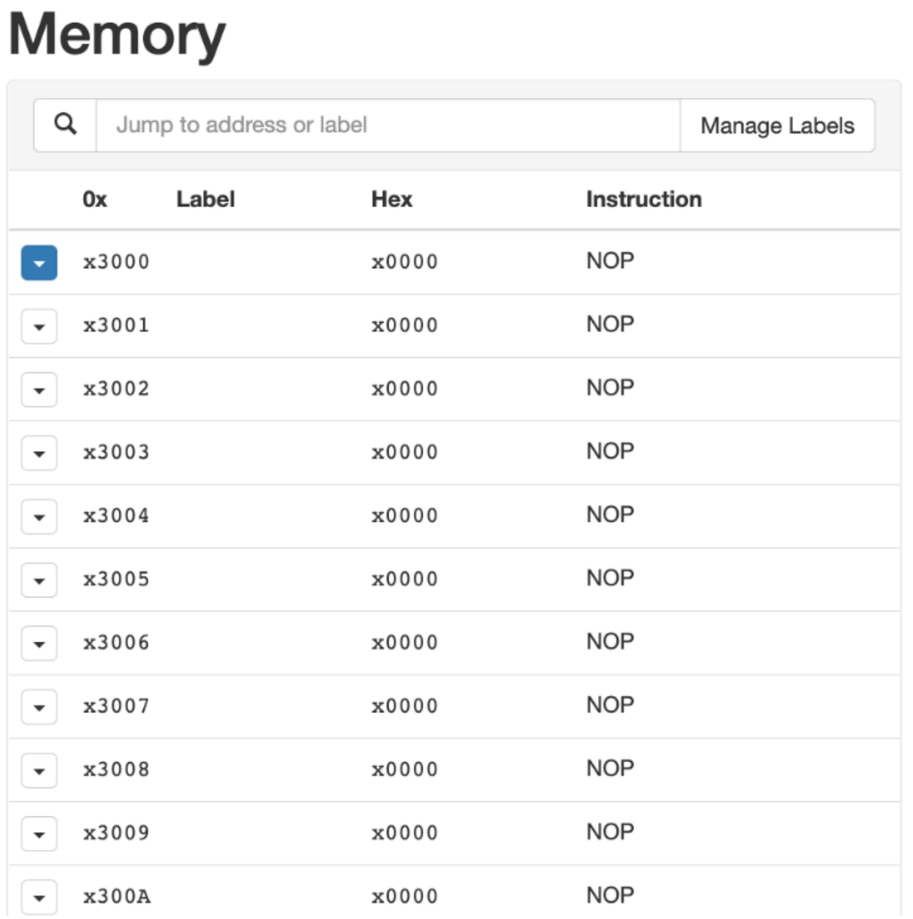
Temporary storage, accessed in a single machine cycle*

- 8 registers
  - R0-R7: General registers
  - PC (program counter), condition codes



Memory

- address space: 216 locations (16-bit addresses)
- addressability: 16 bits



x3000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

ISA: Types of Instruction

- 1. Operate Instructions
  - process data (addition, logical operations, etc.)
- 2. Data Movement Instructions …
  - move data between memory locations and registers.
- 3. Control Instructions …
  - change the sequence of execution of instructions in the stored program.
    - The default is sequential execution: the PC is incremented by 1 at the start of every Fetch, in preparation for the next one.
    - Control instructions set the PC to a new value during the Execute phase, so the next instruction comes from a different place in the program. This allows us to build control structures such as loops and branches.

Six phases of the complete Instruction Cycle

For LC-3

- **Fetch**: load IR with instruction from memory
- **Decode**: determine action to take (set up inputs for ALU, RAM, etc.)
- Evaluate address: compute memory address of operands, if any
- Fetch operands: read operands from memory or registers
- **Execute**: carry out instruction
- **Store** results: write result to destination (register or memory)

Instruction Set

15 Opcodes

- Operate instructions: **ADD, AND, NOT**
- Data movement instructions: **LD, LDI, LDR, LEA, ST, STR, STI**
- Control instructions: **BR, JSR/JSRR, JMP, RTI, TRAP**

- *some opcodes set/clear condition codes, based on result:
- N = negative, Z = zero, P = positive (> 0)

Addressing Modes

How is the location of an operand specified?

- non-memory addresses:
  - immediate
  - register
- memory addresses:
  - PC-relative
  - indirect
  - base+offset

Things need to consider…

What are the key differences between coding on low level languages and high level languages?

Key Concepts in Assembly

- How to store and use variables?
  - Registers

- How to weave control flow?
  - Mostly linear/sequential
  - Simple loops and branches

Previously of CSCI 260…

…And your quiz ;) -Write the keywords related to the red words

- Overview of Little Computer 3(LC-3)
  - Instruction(How many?)
  - Memory(R_____ and M_____)
  - Datatypes(How many bits?)
- 4-6 Steps in Instruction Cycle
- Why do we need more cores, and why do we need more threads
- Addressing Modes of LC-3

Quiz on the Back

- How does MUX and a series of switches make instructions work?
- What is the relationship between a Finite State Machine and an Instruction?
- What are the inputs to an ALU?

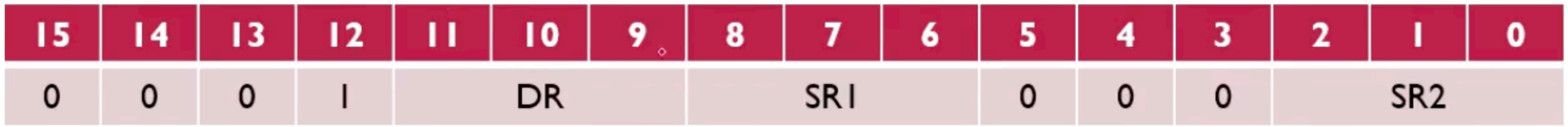
Instructions of LC-3 (Continue)

An example of ADD

Operation and data can fit in 16-bit wide space:

“x1042”

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	0	0	0	0	1	0	0	0	0	1	0



- Number to ‘ADD’ department: 0001
- Destination Register(DR) = Source Register 1(SR1) + Source Register 2(SR2)

ADD / +

Opcode: 0001

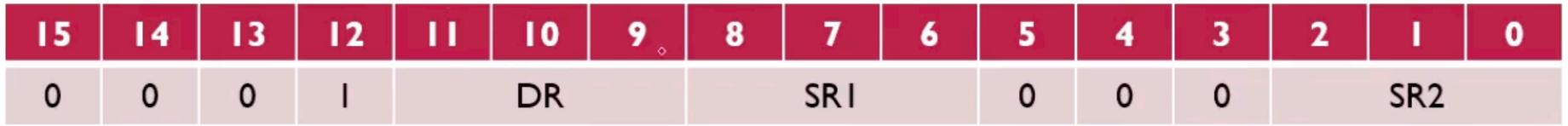
- Destination Register(DR) = Source Register 1(SR1) + Source Register 2(SR2)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			0	0	0	SR2		

ADD

Opcode: 0001

- Destination Register(DR) = Source Register 1(SR1) + Source Register 2(SR2)



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			0	0	0	SR2		

- Destination Register(DR) = Source Register 1(SR1) + 5-bit Immediate Number

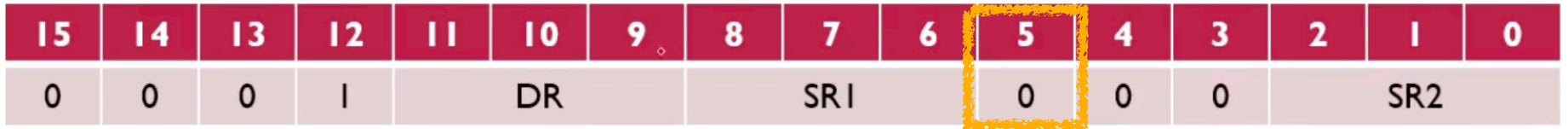


15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			1	Imm5				

ADD

Opcode: 0001

- Destination Register(DR) = Source Register 1(SR1) + Source Register 2(SR2)



- Destination Register(DR) = Source Register 1(SR1) + 5-bit Immediate Number



Switch on bit 5:
0 for the registers
1 for the numbers

An Example of the ADD Revisited

Opcode	DR	SR1	[5:3]	SR2
0001	010	011	000	100
0001 0100 1100 0100				
x14C4				

An Example of the ADD Revisited

Opcode	DR	SR1	[5:3]	SR2
0001	010	011	000	100
0001 0100 1100 0100				
x14C4				

- Opcode: 0001 = ADD
- Destination Register(DR): 010 = R2
- Source Register 1(SR1): 011 = R3
- Source Register 2(SR2): 100 = R4
- Bit 5 is 0: Add between Registers

ADD R2 R3 R4 or R2 <- R3 ADD R4

AND / &

Opcode: 0101

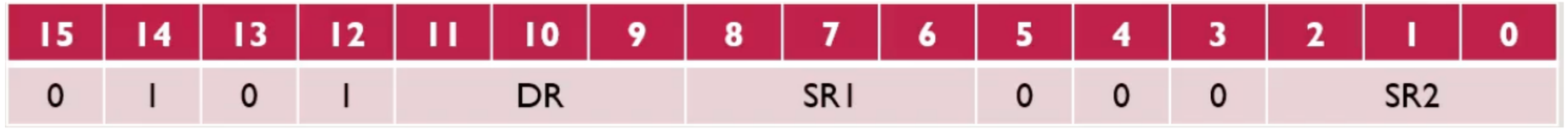
- Destination Register(DR) =
  Source Register 1(SR1) AND Source Register 2(SR2)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			0	0	0	SR2		

AND / &

Opcode: 0101

- Destination Register(DR) =
  Source Register 1(SR1) AND Source Register 2(SR2)



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			0	0	0	SR2		

- Destination Register(DR) =
  Source Register 1(SR1) AND 5-bit Immediate Number

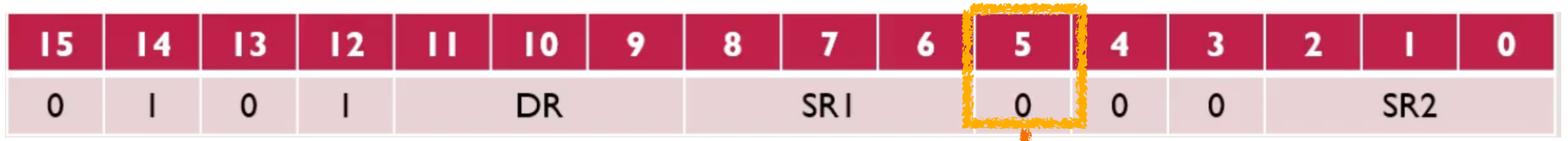


15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			1	Imm5				

AND / &

Opcode: 0101

- Destination Register(DR) =
  Source Register 1(SR1) AND Source Register 2(SR2)



- Destination Register(DR) =
  Source Register 1(SR1) AND 5-bit Immediate Number



Switch on bit 5:
0 for the registers
1 for the numbers

An Example of the AND

Opcode	DR	SR1	[5]	Imm5
0101	110	000	1	11100
0101 1100 0010 1100				
x5C2C				

- Opcode: 0101 = AND
- Destination Register(DR): 110 = R6
- Source Register 1(SR1): 000 = R0
- Immediate Number: 11100 = 11100 in binaries
- Bit 5 is 1: And between a Register and a number.

AND R6 R0 0b11100 or R6 <- R0 AND 0b11100

LD

Opcode: 0010

- Destination Register(DR) =
  the content of memory <PCoffset9> lines away from the Program Counter

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	DR			PCoffset9								

- Example:

Opcode	DR	PCoffset9
0010	001	0 0000 0011
0010 0010 0000 0011		
x2203		

An Example of the LD

Opcode	DR	PCoffset9
0010	001	0 0000 0011
0010 0010 0000 0011		
x2203		

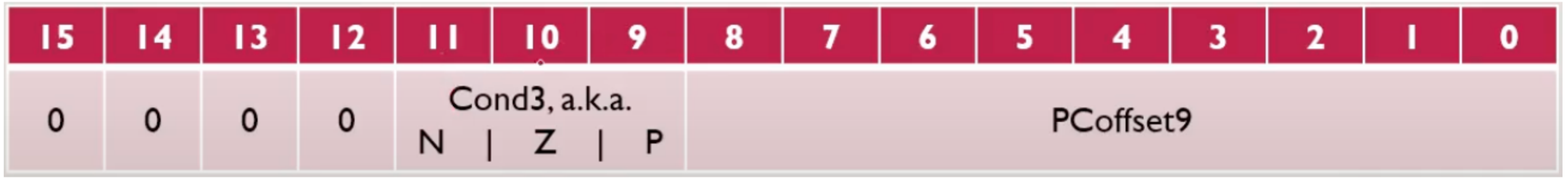
- Opcode: 0010 = LD
- Destination Register(DR): 001 = R1
- PCoffset9: 0 0000 0011 = 3 lines after current Program Counter
- (If PC = x3017, Memory[x301A] = 5, then R1 is loaded 5 after LD instruction.

LD R1 3 or R1 <- M[PC + 3]

BR(Branch)

Opcode: 0000

- If (the output from ALU gave us a Negative / Zero / Positive result), we set the PC to <PCoffset9> of our current position
-



BR(Branch)

Opcode: 0000

- If (the output from ALU gave us a Negative / Zero / Positive result), we set the PC to <PCoffset9> of our current position

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	Cond3, a.k.a. N | Z | P			PCoffset9								

N	Z	P	Value is…
0	0	0	
0	0	1	
0	1	0	= 0
0	1	1	
1	0	0	
1	0	1	!= 0
1	1	0	
1	1	1	

An Example of the BR

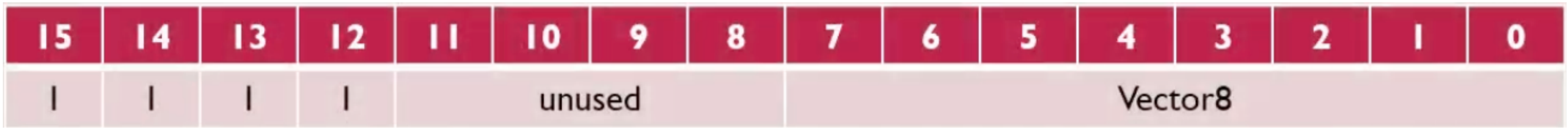
Opcode	Cond3			PCoffset9
0000	1	0	1	1 1111 1110
0000 1011 1111 1110				
x0BFE				

- Opcode: 0000 = BR
- Cond3: N is 1, Z is 0, P is 1 = Check if result is not 0
- PCoffset9: 1 1111 1110 = 2 lines before current Program Counter
- (If PC = x3017, ALU gives a 1, then we are jump to x3015.

TRAP

Opcode: 1111

- “Run program at the index of [Vector8]”



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	unused				Vector8							

- Program at x0025 is the ‘HALT’ program.

Opcode		Vector8
1111	0000	0010 0101
1111 0000 0010 0101		
xF025		

= x0025

= HALT the processor

More Instructions

Operate	Data Movement	Control
ADD	LD	BR
AND	**LDI**	TRAP
NOT	**LDR**	**JMP**
LEA	**ST**	
	STI	
	STR	

Assignment 0:
Complete your surviving package of LC-3 and Assembly

Building Survival Kit

I’ll check your draft next Monday~

- Conversion Table between Binaries, Decimals, Hexadecimal.
- Quick lookup tables for ASCII
- Structure of LC-3 (Datapath Diagram)
- 15 Instructions and Examples
- State Trackers (for Memory and Registers)

RISC and SISC

Reduced or Complex Instructions?

CISC: Do More Per-Instruction

- Memory was expensive and slow in early computer systems, so fewer instructions meant smaller programs that fit better in memory.
- Complex instructions reduced the need for compilers to generate long sequences of simple instructions.
- Designed to be closer to human-readable machine instructions (helpful for assembly programmers).

Reduced or Complex Instructions?

RISC: Do Less, but Do it FAST

- Focus on simplifying hardware for speed and efficiency.
- Encourage pipelining, where multiple instructions are overlapped in execution.
- Optimized for compilers that can generate efficient sequences of simple instructions.

RISC vs SISC

Compare LC-3 with x86

- What are the features between RISC(LC-3) and SISC(x86)

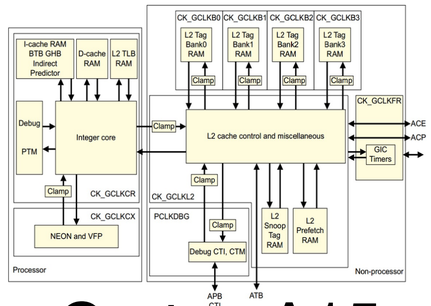
	LC-3	X86
# instructions	15	>1000
# addressing modes	5	>24
Length of instructions	Fixed (4 bytes)	Varied (1-15 bytes)
Operand position	Consistent	Changes
Clock speed	Faster	Slower
# instructions/program	Higher	Lower

Conclusion

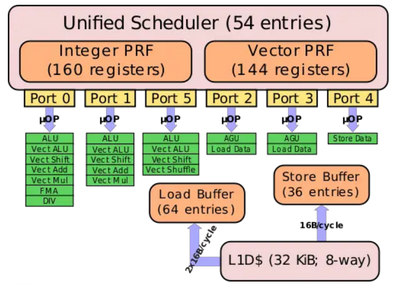
- CISC: Minimize the number of instructions per program, even if each instruction is complex and takes multiple cycles.
- RISC: Minimize the number of cycles per instruction, even if this means more instructions per program.

RISC

CISC



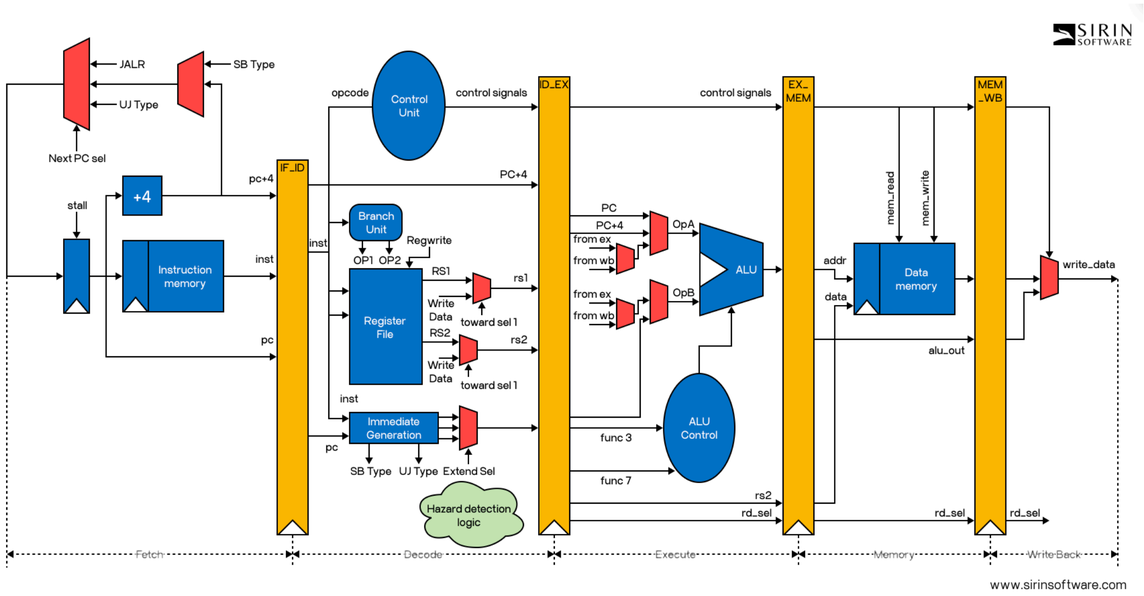
Cortex A15

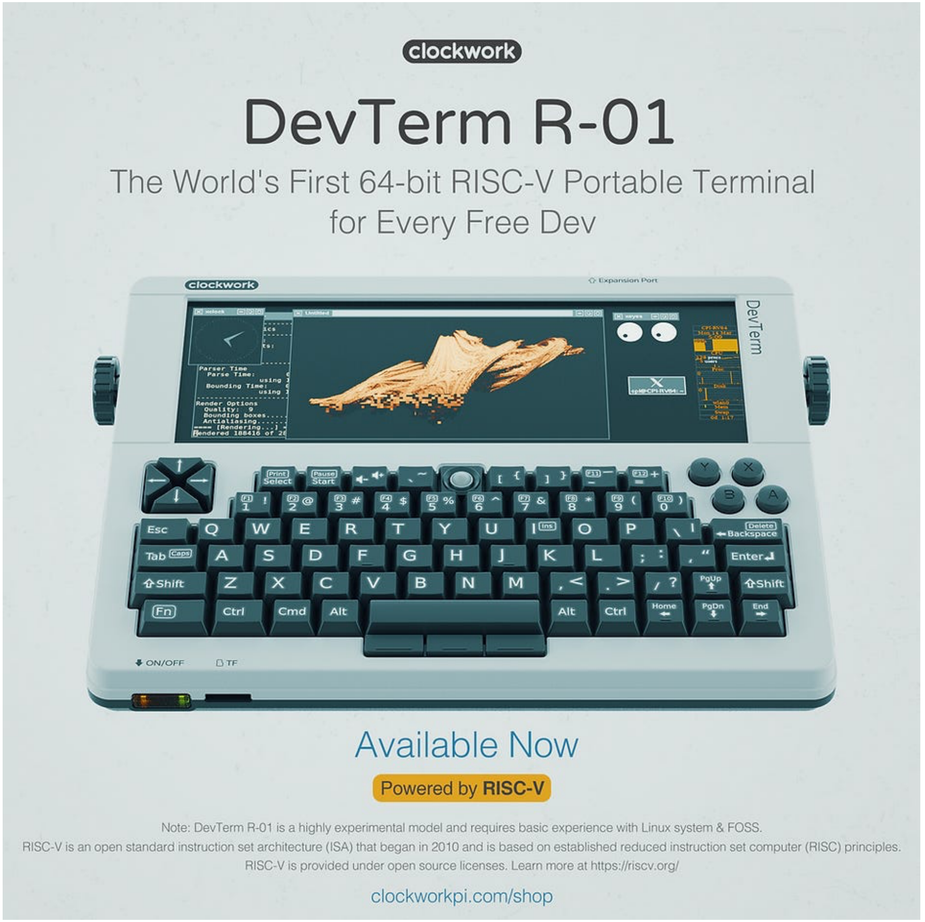


Sandy Bridge

Case Study: RISC-V

Open Sourced RISC Architecture





Case Study: RISC-V

Open Sourced RISC Architecture

RISC-V vs ARM

Feature	RISC-V	ARM
Open Source	Yes, fully open-source ISA	Proprietary, requires licensing from Arm Ltd.
Customization	Highly customizable with support for custom extensions	Limited customization, extensions are predefined by ARM
Power Consumption	Less than 1 Watt of Power	Less than 4 watts of power
Number of General Purpose and Floating Point Registers	32	31
Market Focus	Emerging in embedded systems, AI, and automotive	Dominant in mobile devices, tablets, and embedded systems
Instruction Set	Fixed-length (32-bit) with variable-length extensions	Fixed-length (32-bit/64-bit) with some variable-length instructions
Performance	Optimized for specific applications, can be tailored for performance	Generally high performance, especially in mobile and low-power applications

Case Study: RISC-V

Open Sourced RISC Architecture

RISC-V vs x86

Feature	RISC-V	x86
Architecture Type	RISC (Reduced Instruction Set Computing)	CISC (Complex Instruction Set Computing)
Open Source	Yes, fully open-source ISA	Proprietary, controlled by Intel and AMD
Instruction Complexity	Simple instructions, typically one cycle	Complex instructions can take multiple cycles
Market Focus	Emerging in embedded systems, AI, and automotive	Dominant in PCs, laptops, and servers
Customization	Highly customizable with support for custom extensions	Limited customization; primarily fixed architecture
Power Consumption	Generally lower power consumption, suitable for a variety of applications	Higher power consumption, optimized for performance
Ecosystem	A growing ecosystem with increasing support from tech giants	Established ecosystem, widely supported by software and tools